

75.29/95.06/TB024 Teoría de Algoritmos

Trabajo Práctico N°2

Grupo N° 4

Apellido, Nombres	Email	Padrón
Tania Friedenberger	tfriedenberger@fi.uba.ar	108823
Marco Tosi	mtosi@fi.uba.ar	107237
Payero Juan Ignacio	jpayero@fi.uba.ar	113195
Bloise Julieta	jbloise@fi.uba.ar	98592
Guillermo Hahn	ghahn@fi.uba.ar	112761

Fecha de 1er entrega:	21/06/2026
------------------------------	------------

Observaciones:

Índice

Índice	1
Problema 1: Programación Lineal	2
1. Supuestos	2
2. Variables y Constantes	2
3. Modelo matemático	3
4. Diseño y Estructuras de Datos	3
5. Análisis de Complejidad Computacional	4
5. Solución obtenida	4
Problema 2: Redes de Flujo	4
1. Supuestos, condiciones y limitaciones	4
2. Diseño de la red de flujo	5
3. Seguimiento	7
4. Análisis de Complejidad	9
5. Sets de Datos	9
6. Tiempos de Ejecución	10
7. Informe de Resultados	10
Problema 3: Algoritmos de Aproximación	11
1. Contraejemplo del algoritmo original	11
2. Algoritmo de aproximación propuesto	11
3. Supuestos	11
4. Diseño y demostración de la garantía	12
5. Seguimiento	13
6. Análisis de Complejidad	14
7. Datasets, tiempos e informe de resultados	14
8. Conclusión	15
Problema 4: Algoritmos Aleatorios	15
1. Supuestos	15
2. Diseño	16
3. Seguimiento:	16
4. Análisis de Complejidad	17
5. Demostración	18
6. Sets de Datos	19
7. Informe de Resultados	20

Problema 1: Programación Lineal

1. Supuestos

Asumimos las siguientes condiciones:

1. **Indivisibilidad de las ofertas:** Las ofertas se aceptan en su totalidad o se rechazan. No se puede fraccionar (por ejemplo, no podemos darle 15 paradas al Cliente A por USD 25000).
2. **Homogeneidad de las paradas:** El modelo solo evalúa la cantidad total de paradas asignadas, no la ubicación de las mismas.
3. **Independencia:** Salvo las restricciones explícitas (competencia entre A y D, u opciones excluyentes de B), la aceptación de un cliente no condiciona a los demás.

2. Variables y Constantes

Sea $I = \{A, B_1, B_2, C, D, E, F, G\}$

Para cada oferta $i \in I$ se establecen las siguientes **constantas globales**:

- O_i = Oferta económica (USD) que paga la opción i .
- P_i = Cantidad de paradas de colectivo que solicita ocupar la opción i .

Tabla de los parámetros del problema:

Índice de Oferta ($i \in I$)	Beneficio Ofrecido (O_i)	Paradas Requeridas (P_i)
A (Cliente A)	USD 50.000	30
B_1 (Cliente B, Opción 1)	USD 100.000	80
B_2 (Cliente B, Opción 2)	USD 120.000	120
C (Cliente C)	USD 100.000	75
D (Cliente D)	USD 80.000	50

E (Cliente E)	USD 5.000	2
F (Cliente F)	USD 40.000	20
G (Cliente G)	USD 90.000	100

Dada la naturaleza del problema definimos una variable **binaria**:

- $Y_i \in \{0, 1\}$ = Variable de decisión que toma el valor 1 si se elige la oferta i , y 0 en caso de que se rechace la oferta.

3. Modelo matemático

Función Objetivo:

El objetivo es maximizar el ingreso total recaudado.

- $$\text{Max } Z = \sum_{i \in I} O_i Y_i$$

Restricciones del Sistema:

- **Límite de Infraestructura:** La suma total de las paradas asignadas no puede superar la capacidad máxima disponible en el municipio (200 paradas).

- $$\sum_{i \in I} P_i * Y_i \leq 200$$

- **Exclusión de Opciones Comerciales (Cliente B):** Restricción lógica para asegurar que, como máximo, se adjudique una única opción de las dos ofrecidas por el Cliente B.

- $$Y_{B_1} + Y_{B_2} \leq 1$$

- **Exclusión por Competencia Directa (Clientes A y D):** Restricción de incompatibilidad publicitaria simultánea entre los competidores directos A y D.

- $$Y_A + Y_D \leq 1$$

4. Diseño y Estructuras de Datos

Para modelar y resolver este problema mediante software, se emplearon las siguientes estrategias de diseño en el código:

- **Diccionarios de datos:** Se utilizaron mapas hash para representar las constantes O_i y P_i .

- **Instanciación Dinámica:** Las restricciones y la función objetivo se construyen mediante iteradores algebraicos (pulp.lpSum).

5. Análisis de Complejidad Computacional

El presente modelo pertenece a la familia de problemas de **Programación Lineal Entera (PLE)**, más concretamente la variante *Knapsack* 0-1 con restricciones lógicas). Se resolvió utilizando el algoritmo exacto de **Branch and Bound**.

- **Complejidad Teórica del Peor Caso:** Está dado por la complejidad temporal de **Branch and Bound**. Para n variables de decisión, el límite asintótico temporal es marcadamente exponencial: $O(2^n)$

5. Solución obtenida

La ejecución matemática halló el óptimo global mediante el algoritmo de **Branch and Bound**, arrojando la siguiente política de asignación:

Se deben aceptar y firmar contratos con:

- **Cliente_A** (Usa 30 paradas | Ingreso USD 50000)
- **Cliente_B_Op1** (Usa 80 paradas | Ingreso USD 100000)
- **Cliente_C** (Usa 75 paradas | Ingreso USD 100000)
- **Cliente_E** (Usa 2 paradas | Ingreso USD 5000)

Impacto final: Esta estrategia garantiza un **Ingreso Máximo Bruto de USD 255.000**. El modelo optimizó matemáticamente la combinación respetando las limitantes de exclusividad comercial (al aceptar al Cliente A, se forzó el rechazo del competidor D). En la distribución final, se utilizarán de manera efectiva **187 de las 200 paradas habilitadas**, dejando un total 13 paradas disponibles las cuales resultan insuficientes para alojar ninguna otra propuesta.

Problema 2: Redes de Flujo

1. Supuestos, condiciones y limitaciones

Supuestos

1. La matriz de distancias d es conocida de antemano y está completa, es decir, para cada par de antenas i, j se conoce la distancia $d[i][j]$.
2. Las distancias no son negativas.
3. Una antena no puede ser backup de sí misma.
4. Los parámetros D , k y b son enteros positivos.
5. Las capacidades de la red son enteras, por lo que Edmonds-Karp devolverá un flujo máximo entero y cada unidad de flujo podrá interpretarse como una asignación concreta de backup.

Condiciones

1. Cada antena debe tener al menos k antenas alcanzables dentro de la distancia D . Si alguna antena posee menos de k candidatos posibles, la solución es imposible.
2. La capacidad total disponible debe ser suficiente para satisfacer la demanda total:

$$n * b \geq n * k \Rightarrow b \geq k$$

De lo contrario no existe una solución.

La condición $nb \geq nk$, equivalente a $b \geq k$, es necesaria pero no suficiente. También debe existir una distribución compatible de las asignaciones, condición que se verifica mediante el flujo máximo.

Limitaciones

1. El algoritmo únicamente determina si existe una asignación válida y, en caso afirmativo, devuelve una de ellas. No optimiza ningún otro criterio.
2. La resolución supone que todas las asignaciones de backup tienen la misma importancia. La distancia únicamente se utiliza como indicador de posible backup, y no como un costo a minimizar.
3. La complejidad depende del algoritmo de flujo máximo utilizado. En esta implementación se utiliza Edmonds-Karp para garantizar complejidad polinomial.

2. Diseño de la red de flujo

A) Modelado en red de flujo

Construcción de la red de flujo

El problema se modela con un grafo dirigido, con capacidades donde se representan las antenas y las posibles asignaciones de backup mediante una estructura:

Se construyen los siguientes nodos:

- Una fuente S
- Un conjunto de nodos L_i uno por cada antena i , que representan la **demanda de backups**
- Un conjunto de nodos R_j , uno por cada antena j , que representan la **capacidad de ser backup**
- Un sumidero T

Construcción de aristas

1. Fuente a nodos de demanda de backup

Arista: $S \rightarrow L_i$

Capacidad: k ; la cantidad de backups que debe tener para cumplir la condición del ejercicio

2. Compatibilidad entre antenas

Arista: $L_i \rightarrow R_j$

si y sólo si:

- $i \neq j$ (i distinto de j)
- $d[i][j] < D$ (distancia de i a j menor a la distancia máxima)

Capacidad: 1; una antena no se puede asignar más de una vez a otra como backup.

3. Nodos de backup hacia el sumidero

Arista: $R_j \rightarrow T$

Capacidad: b ; cada antena puede aparecer como backup de a lo sumo b antenas.

Interpretación del flujo max

Cada unidad de flujo representa una asignación concreta de backup. Si existe flujo en la arista $L_i \rightarrow R_j$, entonces la antena j es utilizada como backup de la antena i . Asimismo, como cada antena requiere k backups, el flujo máximo debe ser de $n \cdot k$ para que exista una solución al ejercicio

B) Pseudocódigo y Estructuras de Datos

Estructuras de datos

Se utiliza una **lista de adyacencia** para representar el grafo residual:

Cada nodo almacena una lista de aristas con:

- destino
- capacidad residual
- flujo actual
- referencia a la arista inversa (para el grafo residual)

Representación de una arista

Vértice:

hacia	→ nodo destino
capacidad	→ capacidad máxima
flujo	→ flujo actual
rev	→ arista inversa

Construcción del grafo

para cada i en antenas:

agregar arista $S \rightarrow L[i]$ con capacidad k

para cada par (i, j) :

si $i \neq j$ y $d[i][j] < D$:

agregar arista $L[i] \rightarrow R[j]$ con capacidad 1

para cada j en antenas:

agregar arista $R[j] \rightarrow T$ con capacidad b

Algoritmo de flujo máximo (Edmonds-Karp)

maxFlow = 0

while existe camino aumentante (BFS en grafo residual):

calcular capacidad mínima del camino (cuello de botella)

for each arista del camino:

aumentar flujo en esa arista

disminuir flujo en la arista inversa

maxFlow += cuello de botella

return maxFlow

Recuperación de solución

for each i:

for each vértice saliente de L[i]:

if vértice va hacia R[j] y flujo == 1:

j es backup de i

Condición de éxito

if maxFlow == n * k:

existe solución

else:

no existe solución

3. Seguimiento

Datos de entrada

- n=3 antenas
- k=1
- b=2
- D=5

Matriz de distancias

0 2 6

2 0 3

6 3 0

Construcción de la red

Nodos:

- Fuente: S
- Demanda: L0,L1,L2
- Backup: R0,R1,R2
- Sumidero: T

Aristas

Fuente → Demandante de backup

S → L0 (capacidad k = 1)

S → L1 (capacidad k = 1)

S → L2 (capacidad k = 1)

Vemos cuáles nodos son compatibles:

$d[0][1] = 2$ Si

$d[1][0] = 2$ Si

$d[1][2] = 3$ Si

$d[2][1] = 3$ Si

$d[0][2] = 6$ No

$d[2][0] = 6$ No

Demandante → Backup

$L0 \rightarrow R1$

$L1 \rightarrow R0$

$L1 \rightarrow R2$

$L2 \rightarrow R1$

(todas con capacidad 1)

Backup → Sumidero

$R0 \rightarrow T$

$R1 \rightarrow T$

$R2 \rightarrow T$

(todas con capacidad $b = 2$)

Flujo (Karp):

Iteración 1

Camino encontrado:

$S \rightarrow L0 \rightarrow R1 \rightarrow T$

Flujo enviado: 1

Iteración 2

$S \rightarrow L1 \rightarrow R0 \rightarrow T$

Flujo enviado: 1

Iteración 3

$S \rightarrow L2 \rightarrow R1 \rightarrow T$

Flujo enviado: 1

Flujo total = 3

Flujo necesario: $n * k = 3 \Rightarrow$ Hay solución

Reconstrucción de solución

Revisamos aristas $L_i \rightarrow R_j$ con flujo 1:

$L0 \rightarrow R1$

$L1 \rightarrow R0$

$L2 \rightarrow R1$

Resultado

Antena 0 $\rightarrow \{1\}$

Antena 1 $\rightarrow \{0\}$

Antena 2 $\rightarrow \{1\}$

Fin del Seguimiento

4. Análisis de Complejidad

Construcción del Grafo

Vértices

- 1 fuente S
- n nodos Li
- n nodos Ri
- 1 sumidero T

Por lo tanto: $V = 2n+2 = O(n)$

Aristas

1. Fuente → antenas

Una por cada antena: $O(n)$

2. Compatibilidades Li→Rj

En el peor caso (grafo completo bajo el umbral D): $O(n^2)$

3. Antenas → sumidero

Una por cada antena: $O(n)$

Total: $O(n^2)$

Complejidad de Flujo Máximo

Al usarse el algoritmo de Edmonds-Karp, la complejidad resulta: $O(E^2 * V)$

Transformando a parámetros de entrada:

$$O(n^4 * n) \simeq O(n^5)$$

Complejidad de la Recuperación de la Solución:

Dado que recorremos todas las aristas: $O(n^2)$

$$\text{Complejidad Total: } O(n^2 + n^5 + n^2) \simeq O(n^5)$$

5. Sets de Datos

Los sets de datos se encuentran generados en el directorio “datasets”, con el formato:

-Filas/Columnas

-Dato D; Dato k; Dato b

-Matriz de antenas

6. Tiempos de Ejecución

Gráfico de comparación

Complejidad Teórica Calculada VS Complejidad del algoritmo en Práctica

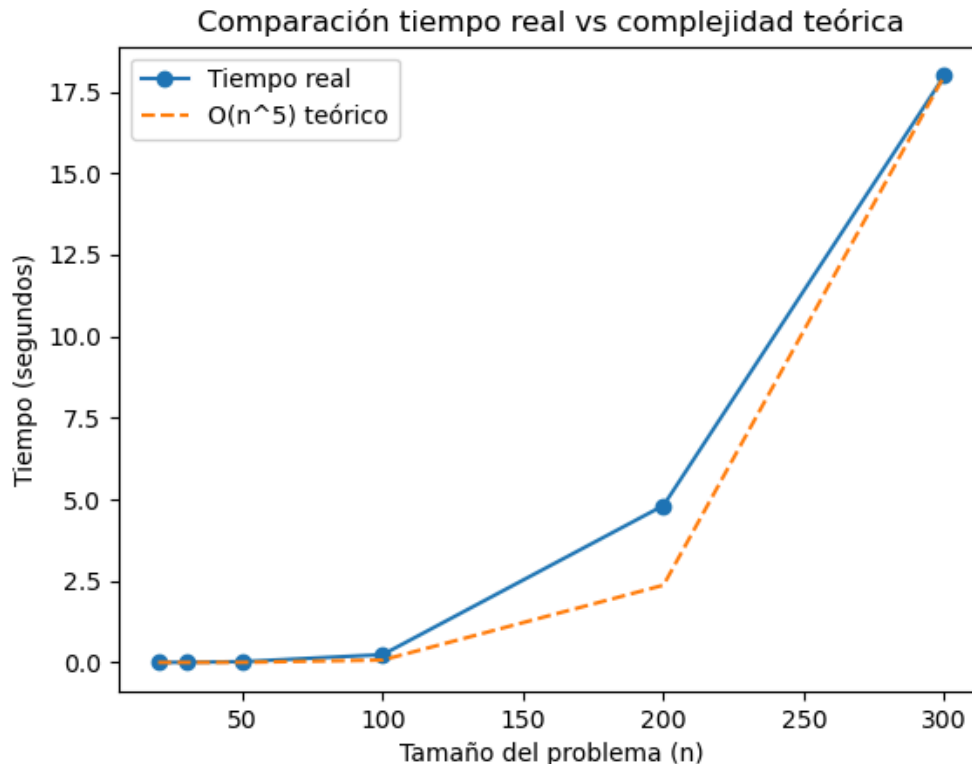


Figura 1. Comparación entre los tiempos medidos y la curva teórica $O(n^5)$

7. Informe de Resultados

Luego de realizar mediciones de tiempo del algoritmo propuesto utilizando distintos tamaños de dataset (desde 20 a 200 columnas/filas), los resultados obtenidos se compararon con la curva teórica de una complejidad $O(n^5)$, correspondiente a la complejidad deducida en el punto 4.

En el gráfico comparativo puede observarse que los tiempos experimentales presentan una tendencia muy similar a la curva teórica. Para tamaños pequeños, los tiempos de ejecución son reducidos así como las diferencias entre ambas curvas.

Sin embargo, a medida que aumenta el tamaño del dataset (aumentan las antenas) el crecimiento del tiempo de ejecución se vuelve más pronunciado, siguiendo lógicamente una curva de $O(n^5)$. Particularmente para los casos de mayor tamaño ($n=100$, 200 y 300), se aprecia claramente que el incremento del tiempo medido es consistente con el crecimiento esperado teóricamente.

Las pequeñas diferencias observadas entre ambas curvas pueden atribuirse a factores como:

- Costos constantes no considerados en el análisis asintótico.
- Implementación específica de las estructuras de datos.
- Variaciones del sistema operativo.

- etc.

Los tiempos medidos presentan un crecimiento compatible con la tendencia utilizada como referencia. El comportamiento observado del algoritmo se corresponde con la complejidad temporal $O(n^5)$ estimada a partir del uso de Edmonds-Karp sobre una red de flujo con $O(n)$ vértices y $O(n^2)$ aristas.

Problema 3: Algoritmos de Aproximación

1. Contraejemplo del algoritmo original

El algoritmo del enunciado recorre A en el orden dado y toma cada elemento si cabe.

Consideramos la instancia:

$$A = \{1, 10\}, B = 10$$

Paso	Elemento	T antes	T + a	¿Entra?	T después
1	1	0	1	si	1
2	10	1	11	no	1

El algoritmo devuelve $S = \{1\}$, $\text{suma} = 1$.

Sin embargo, el subconjunto $\{10\}$ es factible ($10 \leq 10$) y tiene $\text{suma} = 10$.

Verificación: $1 < 10/2 = 5$ La suma del greedy es menos de la mitad de la suma de otro subconjunto factible.

¿Por qué falla?: el elemento pequeño (1) es tomado primero e impide agregar el elemento grande (10).

2. Algoritmo de aproximación propuesto

Idea: El contraejemplo muestra que el orden de recorrido puede provocar soluciones arbitrariamente malas. Para obtener la garantía solicitada, se propone ordenar los elementos de mayor a menor.

El código completo se incluye en el archivo `problema3.py`.

3. Supuestos

- Todos los elementos de A son **enteros positivos** (≥ 1).
- B es un **entero positivo**.
- Los elementos de A pueden ser mayores que B ; simplemente nunca son elegidos (no afectan OPT).
- A puede estar vacío; en ese caso $\text{OPT} = 0$ y el algoritmo devuelve $S = \{\}$ correctamente.
- No se asume ninguna distribución particular de los valores; la garantía vale para cualquier instancia.

4. Diseño y demostración de la garantía

a) Demostración de la garantía: $\text{suma}(S) \geq \text{OPT}/2$

Sea $A = [a_1 \geq a_2 \geq \dots \geq a_n]$ el arreglo ya ordenado (los elementos $> B$ se ignoran pues no pueden estar en ningún subconjunto factible). Sea OPT la suma máxima de cualquier subconjunto factible; en particular $\text{OPT} \leq B$.

Caso 1: el greedy toma todos los elementos.

$$\text{suma}(S) = \text{suma}(A) \geq \text{OPT}.$$

Caso 2: existe al menos un elemento rechazado. Sea a_i el primero que el greedy no puede agregar, y sea T_i el acumulado en ese momento.

Por definición de rechazo:

$$T_i + a_i > B \geq \text{OPT} \quad (i)$$

Como a_1 es el mayor elemento de A y cabe en B ($a_1 \leq B$) con $T = 0$ al inicio, el greedy **siempre** toma a_1 . Por tanto:

$$T_{\text{final}} \geq a_1 \geq a_i \quad (ii)$$

$$T_{\text{final}} \geq T_i \quad (iii) \quad (T \text{ solo crece})$$

Sumando (ii) y (iii):

$$2 \cdot T_{\text{final}} \geq T_i + a_i > \text{OPT} \quad [\text{por (i)}]$$

$$\therefore T_{\text{final}} > \text{OPT}/2$$

La garantía de $1/2$ es ajustada: con $A = \{B/2 + 1, B/2, B/2\}$ (B par), el greedy toma solo $B/2 + 1 \approx B/2$ mientras $\text{OPT} = B$ (los dos $B/2$). La razón tiende a $1/2$ desde arriba.

b) Pseudocódigo

greedy_aprox(A, B)

$A_{\text{ord}} \leftarrow \text{ordenar } A \text{ de mayor a menor} \quad // O(n \log n)$

$S \leftarrow \{\}$

$T \leftarrow 0$

Para cada a *en* A_{ord} : $// O(n)$

Si $T + a \leq B$:

$S \leftarrow S \cup \{a\}$

$T \leftarrow T + a$

fin si

fin para

devolver S

c) Estructura de datos

- **A_ord:** arreglo de enteros ordenado (creado con sort estable).
- **T:** entero acumulador de la suma actual.
- **S:** lista de los elementos seleccionados.

5. Seguimiento

Ejemplo del enunciado: $A = \{9, 3, 5\}$, $B = 13$

Ordenado: [9, 5, 3]

Paso	a	T antes	T + a	¿Entra?	T	S
1	9	0	9	sí	9	{9}
2	5	9	14	no	9	{9}
3	3	9	12	sí	12	{9, 3}

Resultado: $S = \{9, 3\}$, suma = 12.

OPT (DP) = 12. Ratio = 1.0

Contraejemplo resuelto: $A = \{1, 10\}$, $B = 10$

Ordenado: [10, 1]

Paso	a	T antes	T + a	¿Entra?	T	S
1	10	0	10	sí	10	{10}
2	1	10	11	no	10	{10}

Resultado: $S = \{10\}$, suma = 10 = OPT.

(El greedy ordenado resuelve exactamente el caso donde el original fallaba.)

Ejemplo adicional: $A = \{7, 5, 4, 3\}$, $B = 10$

Ordenado: [7, 5, 4, 3]

Paso	a	T antes	T + a	¿Entra?	T	S
1	7	0	7	sí	7	{7}
2	5	7	12	no	7	{7}
3	4	7	11	no	7	{7}
4	3	7	10	sí	10	{7, 3}

Resultado: $S = \{7, 3\}$, suma = 10 = OPT.

Ejemplo con elementos mayores que B: $A = \{25, 15, 9, 5, 3, 2\}$, $B = 10$

Ordenado: [25, 15, 9, 5, 3, 2]. Los elementos 25 y 15 superan B, por lo que el greedy los descarta al intentar agregarlos.

Paso	a	T antes	T + a	¿Entra?	T	S
1	25	0	25	no ($> B$)	0	{}
2	15	0	15	no ($> B$)	0	{}
3	9	0	9	sí	9	{9}
4	5	9	14	no	9	{9}
5	3	9	12	no	9	{9}
6	2	9	11	no	9	{9}

Resultado: $S = \{9\}$, suma = 9.

OPT = {5, 3, 2}, suma = 10.

Verificación de la garantía: $9 \geq 10/2 = 5$

Verificación con la demostración:

- $a_{\square} = 5$ (primer rechazado), $T_{\square} = 9$
- $T_{\square} + a_{\square} = 14 > B = 10 \geq \text{OPT} = 10$
- $T_{\text{final}} = 9 \geq a_{\text{válido}} = 9 \geq a_{\square} = 5$
- $2 \cdot T_{\text{final}} = 18 > \text{OPT} = 10 \rightarrow T_{\text{final}} = 9 > 5 = \text{OPT}/2$

Este ejemplo ilustra que el algoritmo no siempre encuentra el óptimo (devuelve 9 en lugar de 10), pero la garantía de aproximación se cumple. La razón aquí es $9/10 = 0.9$.

6. Análisis de Complejidad

Paso	Complejidad
Ordenar A	$O(n \log n)$
Recorrido greedy	$O(n)$
Total	$O(n \log n)$

La complejidad está dominada por el ordenamiento. El espacio adicional es $O(n)$ para el arreglo ordenado y la lista S.

7. Datasets, tiempos e informe de resultados

Los resultados completos se encuentran en la salida del script (`python3 problema3.py`) y el gráfico en “Imagen 1”.

Para $n \leq 2000$ se calculó el óptimo exacto mediante programación dinámica ($O(n \cdot B)$) y se verificó que $\text{suma_aprox} \geq \text{OPT}/2$ en todos los casos. Para $n > 2000$ el DP es impracticable para estos tamaños — con $n = 100.000$ y $B = 10.000$ requeriría 10^9 operaciones, lo que en Python implica varios minutos de ejecución frente a los 0.018 segundos del algoritmo aproximado — por lo que solo se reporta la suma aproximada.

En la práctica, la razón $\text{suma_aprox}/\text{OPT}$ fue siempre superior a 0.99 para los datasets aleatorios, lo que muestra que el algoritmo es excelente en promedio aun cuando la garantía teórica sólo asegura 0.5.

Complejidad empírica

El cociente $t / (n \cdot \log n)$ se mantiene aproximadamente constante para todos los valores de n , lo que confirma que el tiempo de ejecución es efectivamente $\Theta(n \log n)$. El gráfico de la izquierda superpone la curva medida con la curva teórica escalada; el ajuste es muy bueno a partir de $n = 1000$ (zona donde el costo del ordenamiento domina sobre el overhead de Python).

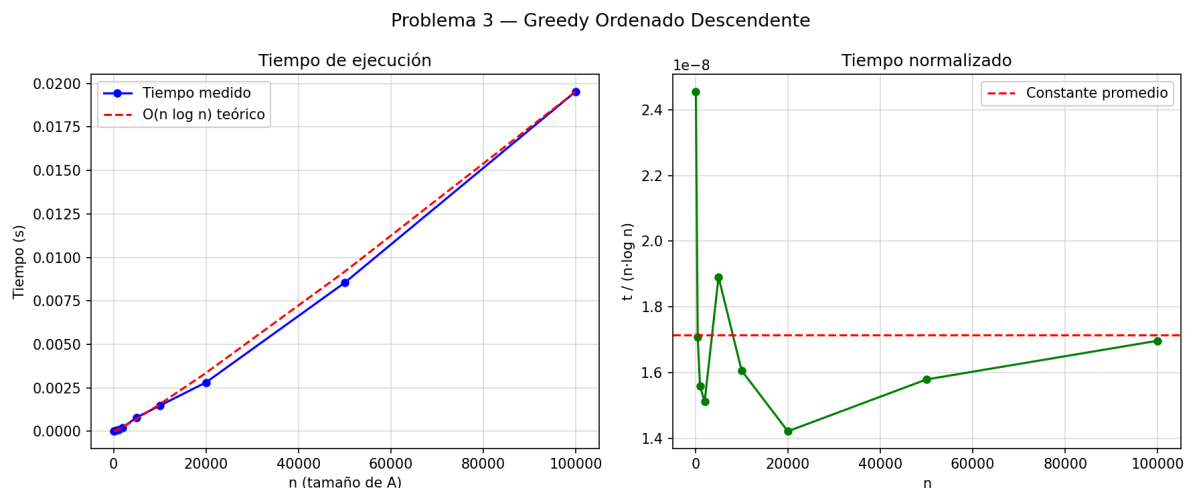


Figura 2. Comparación entre los tiempos medidos y la curva teórica $O(n \log n)$

8. Conclusión

La garantía se cumplió en todos los experimentos realizados. La cota de $(1/2)$ es ajustada asintóticamente para la familia de instancias $(A=\{B/2+1, B/2, B/2\})$, ya que la razón entre la solución obtenida y la solución óptima se aproxima a $(1/2)$ a medida que aumenta (B) . En las instancias aleatorias, el algoritmo obtuvo soluciones prácticamente óptimas.

La **complejidad temporal** es $O(n \log n)$, en concordancia con el análisis teórico.

Problema 4: Algoritmos Aleatorios

1. Supuestos

- El grafo es no dirigido.
- No contiene lazos.

- Cada arista se almacena una única vez.
- Los colores se eligen de manera uniforme e independiente.
- El generador aleatorio puede inicializarse con una semilla para reproducibilidad.

2. Diseño

Dado un grafo $G=(V,E)$, se asigna a cada vértice uno de tres colores posibles elegidos al azar de forma equiprobable e independientemente de los demás vértices.

Una vez coloreados todos los vértices, se recorren las aristas del grafo y se cuenta cuántas quedan satisfechas. Una arista (u,v) se considera satisfecha si los colores asignados a u y v son distintos.

Posteriormente, el algoritmo retorna, el coloreo de los vértices y la cantidad de aristas satisfechas.

Al tratarse de una asignación completamente aleatoria, distintas ejecuciones sobre el mismo grafo pueden producir resultados diferentes.

El algoritmo pertenece al tipo de algoritmos Monte Carlo, el cual es de orden polinomial, con tiempo de ejecución acotado, el algoritmo siempre produce una asignación de uno de tres colores a cada vértice. Sin embargo, la cantidad de aristas satisfechas es aleatoria y puede variar entre ejecuciones.

PseudoCódigo:

```
función tres_coloreo_randomizado(Grafo)
    colores = diccionario vacío
    satisfechas = 0

    for each vértice v en G:
        agregar a colores[v] un color aleatorio entre {1,2,3}

    for each arista (u,v) en G:
        si colores[u] ≠ colores[v]:
            satisfechas = satisfechas + 1

    devolver (colores, satisfechas)
```

Estructuras de Datos Utilizadas:

- Una implementación de Grafo con listas de adyacencia, la cual permite obtener los vértices en tiempo lineal, y recorrer las aristas siguiendo la lista de adyacencia
- Un diccionario de colores que almacena una clave que representa al vértice sobre el que se aplica el color, y un valor que representa el color aplicado

3. Seguimiento:

Datos de Entrada:

- **Vértices (V):** {0,1,2,3}
- **Aristas (E):** {(0,1),(0,2),(1,2),(1,3),(2,3)}
- **Cantidad de aristas (|E|):** 5
- **Cota teórica esperada:** $\frac{2}{3} \times 5 = 3.33$ aristas satisfechas en promedio.

Inicialización:

colores = {} (Diccionario vacío)

aristas_satisfechas = 0

Asignación de Colores:

Vértice	Numero de Color	Diccionario colores
0	1	{0:1}
1	2	{0:1, 1:2}
2	1	{0:1, 1:2, 2:1}
3	3	{0:1, 1:2, 2:1, 3:3}

Verificación de Aristas:

Iteración	Arista (u,v)	color de vértice u	color de vértice v
1	(0,1)	1	2
2	(0,2)	1	1
3	(1,2)	2	1
4	(1,3)	2	3
5	(2,3)	1	3

Salida del Algoritmo:

Diccionario de coloreo: {0: 1, 1: 2, 2: 1, 3: 3}

Aristas satisfechas en esta corrida: 4

Conclusión:

En esta ejecución en particular, el algoritmo tuvo 4 aristas satisfechas (la de iteración 1, 3, 4 y 5). En esta corrida se obtuvieron 4 aristas satisfechas. Este valor particular es superior a $2|E|/3$, aunque la garantía teórica se refiere al promedio esperado sobre múltiples ejecuciones y no a cada ejecución individual.

4. Análisis de Complejidad

Complejidad de Métodos de Grafo:

grafo.obtener_vertices():

Complejidad: $O(|V|)$, ya que debe pasar las claves del diccionario de vértices a una lista.

grafo.obtener_aristas():

Recorre todos los vértices y, para cada uno, recorre sus vecinos. En un grafo no dirigido, añade la arista si no fue visitada usando un conjunto (set).

Complejidad: $O(|V|+|E|)$, ya que visita cada vértice y cada arista exactamente una cantidad constante de veces.

Complejidad de tres_coloreo_randomizado():

En la asignación de colores:

1. grafo.obtener_vertices() toma tiempo $O(|V|)$.
2. El bucle for se ejecuta $|V|$ veces.
3. Dentro del bucle, random.randint(1, 3) y la inserción en el diccionario colores[v] toman tiempo constante $O(1)$.

Complejidad de Asignación: $O((|V|+|V|)*1) \simeq O(|V|)$

En el conteo de aristas satisfechas:

1. grafo.obtener_aristas() toma tiempo $O(|V|+|E|)$.
2. El bucle for se ejecuta exactamente $|E|$ veces (una por cada arista del grafo).
3. Dentro del bucle, las consultas al diccionario colores[u] y colores[v], al igual que la comparación y el incremento de la variable toman tiempo constante $O(1)$.

Complejidad del Conteo: $O((|V|+|E|)+|E|*1) \simeq O(|V|+|E|)$

Complejidad Total:

$O(|V|)+O(|V|+|E|) \simeq O(|V|+|E|)$

5. Demostración

Definición de Variables Aleatorias:

Sea $G=(V,E)$ un grafo no dirigido. Para cada arista $e=(u,v) \in E$, definimos una variable aleatoria indicadora X_e que representa si la arista fue satisfecha por el algoritmo de coloreo aleatorio:

$X_e = \begin{cases} 1 & \text{si } \text{color}[u] \neq \text{color}[v] \\ 0 & \text{si } \text{color}[u] = \text{color}[v] \end{cases}$

Probabilidad de Éxito:

Dado que el algoritmo asigna a cada vértice un color del conjunto $\{1,2,3\}$ de forma independiente y con una distribución uniforme, la probabilidad de que un vértice tome un color específico es exactamente $\frac{1}{3}$.

Si u tiene 3 colores posibles y v también: usando la Regla de Laplace: $\frac{\text{cantidad de casos favorables}}{\text{cantidad de casos totales}}$

casos favorables:

- $u : 1, v : 2$
- $u : 1, v : 3$

- $u : 2, v : 1$
- $u : 2, v : 3$
- $u : 3, v : 1$
- $u : 3, v : 2$

Por lo tanto tenemos 6 casos favorables

casos totales: son 9 tres de u por 3 de v

=> la probabilidad de que una arista sea compatible es de $6/9 = \frac{2}{3} = 0.667$

Valor esperado de la indicadora:

Al ser X_e una variable de distribución Bernoulli, su valor esperado (Esperanza, $E[X_e]$) coincide con su probabilidad de éxito:

$$E[X_e] = P(X_e = 1) = \frac{2}{3}$$

Además, sea X una variable aleatoria que representa el número total de aristas satisfechas en el grafo. Siendo X la suma de las variables indicadoras de todas las aristas, que aplicando el principio de Linealidad de la Esperanza, nos queda

$$X = \sum_{e \in E} X_e \Rightarrow E[X] = E\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} E[X_e] = \frac{2}{3} * |E|$$

(recordando que $|E|$ representa a la cantidad de aristas del grafo

Vinculación con c^* :

Por definición del problema de optimización, c^* representa el número máximo de aristas que pueden ser satisfechas simultáneamente por una asignación óptima de 3 colores.

Como una solución óptima no puede satisfacer más aristas de las que existen en el grafo: $c^* \leq |E|$; luego multiplicamos ambos miembros por la constante $\frac{2}{3}$, manteniendo la desigualdad: $\frac{2}{3} c^* \leq \frac{2}{3} |E|$

Dado que se demostró que el valor esperado del algoritmo es $E[X] = \frac{2}{3} |E|$, se concluye que: $E[X] \geq \frac{2}{3} c^*$

Conclusión:

Queda demostrado Teóricamente que el algoritmo aleatorio propuesto soluciona un 3-Coloreo cuyo número esperado de aristas satisfechas es de al menos $\frac{2}{3} c^*$, cumpliendo con la cota inferior solicitada para cualquier grafo de entrada.

6. Sets de Datos

Los sets de datos generados recibidos por el algoritmo propuesto tienen la forma:

- Primera línea: Cantidad de Vértices
- El Resto del Archivo: asignación de Aristas (u,v) tal que u conecta con v

(los resultados de una ejecución y el promedio de 1000 ejecuciones con cada grafo se encuentra en el directorio resultados_datasets)

7. Informe de Resultados

Comparación de Resultados:

A partir de la ejecución del algoritmo propuesto sobre 4 datasets diferentes, obtenemos los resultados de satisfacción de aristas tanto para una ejecución con cada dataset, como para 1000 ejecuciones con cada dataset con el fin de observar como el promedio de satisfacción se aproxima al calculado teóricamente

Dataset	Vértices	Aristas	Satisfechas en una Corrida	Promedio de Satisfechas en 10.000 corridas	Esperado Teórico	Porcentaje de Desvío
Dataset N°1	5	4	3	2,6648	2.6666	0.07%
Dataset N°2	50	577	385	384,5898	384,6666	0.02%
Dataset N°3	200	992	700	661,2461	661,3333	0.01%
Dataset N°4	400	7893	5266	5322,4202	5322	0.008%

Conclusión:

A partir de los resultados obtenidos notamos que mientras el grafo sea pequeño, la satisfacción de una sola corrida puede alejarse del Esperado, pero al realizar varios intentos, el porcentaje del desvío del Esperado se vuelve menor a 0.01%.

Por otro lado a medida que el grafo es más grande, la satisfacción de un intento se vuelve más cercana al Esperado (en general, pues aún sigue dependiendo del azar), y por ende, el promedio de 10.000 intentos se acerca mucho más al valor teórico, como por ejemplo el promedio del dataset N°4 notamos que el porcentaje de desvío respecto al Esperado es menor al 0.001%

En Conclusión, podemos decir que SI se cumple que el valor de los resultados de ejecución se corresponda con el valor calculado teóricamente